

Chapitre V:

Les sous programmes



- ▶ Les sous programmes en PL/SQL sont des **blocs nommés**, avec paramètres en entrée et/ou sortie
- ▶ Deux types de sous-programmes
 - **Fonction** retourne un **résultat unique**
 - **Procédure** stockée retourne **un ou plusieurs résultats**
- ▶ Un sous programme est défini dans la section DECLARE du bloc.
 - Il s'agit des sous **programme non-stockés** dont l'appel est temporairement limité au bloc où ils sont définis
 - Il est créée par la commande CREATE en tant qu'objet du dictionnaire Oracle : Il s'agit des **fonctions et des procédures stockées ou cataloguées**.
- ▶ Ces sous-programmes seront compilés et stockés dans la base comme tout autre objet manipulé par le moteur du SGBD.



- ▶ **Sécurité** - Droit d'accès sur les sous-programmes et non sur les objets du bloc associé (GRANT EXECUTE ON NomProcedure to Util)
- ▶ **Performance** - Programme compilé et optimisé
 - Réduction du nombre d'appels à la base
 - Partage de code
- ▶ **Productivité** - Simplicité de la maintenance des programmes : modularité, lisibilité, réutilisabilité, etc



Une procédure est un traitement qui peut contenir:

- des **commandes du LMD du SQL**
- des **instructions PL/SQL**
- des **variables**, des **constantes**, des **curseurs**
- un gestionnaire **d'exceptions**

Création des procédures

```
Create [or replace] procedure <nom_procédure> [ (nom_paramètre1  
{IN|OUT|INOUT} type_paramètre1, [paramètre2 {IN|OUT|INOUT}  
type_paramètre2, ...])]  
{IS | AS}  
  -- Déclaration locales de variables, constante, curseur, ...  
BEGIN  
  -- Instructions exécutables  
[EXCEPTION  
  -- Gestion des exceptions ]  
END [nom_procédure] ;
```



CREATE : compile la procédure et stocke son code source dans le dictionnaire Oracle.

Si la compilation aboutit, l'exécutable est également stocké.

Dans ce cas l'appel de l'exécutable se fait comme suit :

```
EXECUTE <nom_procedure> (paramètre 1, paramètre 2, ...);
```



OR REPLACE : supprimer (**DROP**) et recréer le code de la procédure.

- Il faut noter l'absence du mot réservé **DECLARE**. La section déclarative commence directement après **IS** (ou **AS**)
- PL/SQL offre trois modes de passage de paramètres : en entrée (**IN**), en sortie (**OUT**) et en entrée/sortie (**IN OUT**).

► Le mot clé **IN** indique que **la variable est passée en entrée**.

► Le mot clé **OUT** indique qu'elle est **renseignée par la procédure puis renvoyée en sortie** (à l'appelant).

► Le mot clé **INOUT** est une combinaison des deux méthodes : la variable est passée en entrée, renseignée par la procédure puis renvoyée à l'appelant (*passage par référence*).



L'appel des procédures stockées se fait dans les instructions procédurales PL/SQL.

Appel des procédures

```
DECLARE
```

```
...
```

```
BEGIN
```

```
...
```

```
-- Appel d'une procédure
```

```
<nom_procédure> (paramètre 1, paramètre 2, ...);
```

```
...
```

```
END ;
```



```
--exemple  SQL PLUS
set serveroutput on
EXECUTE Inserer LignComm('F45','Estrade', 'C',50,100, 50);
```



**Soit la table : Article (Nart, DesArt, Pu, QteStock, QteMax)
Créer une procédure qui permet de baisser le prix d'un article donné d'un taux donné.**



Le programme suivant illustre l'utilisation de telle procédure :

```
Declare
  x number :=2 ;
  y number :=100 ;
  t number :=5;
Begin
  --appel de la procédure
  reduire_prix(x,t) ;
  reduire_prix(y,t) ;
  reduire_prix(20,t) ;
End ;
```



- ▶ Structure générale analogue à celle d'une procédure mais retourne un résultat unique
- ▶ Retour de résultat avec la clause RETURN
- ▶ Appels à partir :
 - d'une requête SQL
 - d'une procédure, fonction, programme PL/SQL
 - d'un programme externe

```
CREATE [OR REPLACE] FUNCTION <nom_fonction>
[(paramètre, [paramètre...])] return type {IS | AS}
-- Déclaration locales de variables, constant, ...
BEGIN
-- Instructions exécutables
-- la dernière instruction c'est : return (résultat)
[EXCEPTION
-- Gestion des exceptions ]
END [nom_fonction] ;
```

Return dans la définition d'une fonction permet de spécifier le type du résultat retourné par la fonction

Création des fonctions



- L'appel d'une fonction doit retourner un résultat. Les fonctions stockées sont appelées dans les instructions procédurales PL/SQL.
- Elles peuvent être utilisées dans les commandes SQL comme si elles étaient des fonctions intégrées, mais sous certaines conditions.

Appel des fonctions

```
DECLARE
<Paramètre_Réel> <Type_de_Retour_de_la_fonction> ;
BEGIN
  -- Appel d'une fonction dans une expression PL/SQL
  Paramètre_Réel := <nom_fonction> (paramètre 1, paramètre 2, ...);
END ;
```



Contrairement à une procédure, une fonction stockée **peut être appelée dans les commandes SQL** : SELECT, INSERT, UPDATE et DELETE, **partout où l'appel d'une fonction intégrée est valide à condition de respecter les restrictions suivantes** :

- ▶ Les types des paramètres de la fonction doivent être parmi : **CHAR, DATE, NUMBER** et les **type compatibles**.
- ▶ La fonction **ne doit pas** comporter d'ordre INSERT, UPDATE, DELETE.



Soit la table ARTICLE (NART, DesArt, Pu, QtStk, QtMax). Créer une fonction permettant de retourner la quantité à approvisionner d'un article donné.

```
Create function Approvisionner (PNART ARTICLE.NART%TYPE)
Return number
Is
Qte number;
Begin
Select QtMax - QtStk into Qte from ARTICLE where NART = PNART;
Return (QTE);
End;
```

```
Declare
PQTE ARTICLE.QTESTK%TYPE;
Begin
--appel de la fonction dans une expression PL/SQL
PQTE := Approvisionner(10) ;
--appel de la fonction dans une commande SQL
SELECT Approvisionner(10) into PQTE FROM ARTICLE;
End ;
```



```
CREATE OR REPLACE FUNCTION Prix_Moyen
(-----) -----
-----
AvgPrix NUMBER(4,2);

BEGIN
SELECT AVG(Prix) INTO AvgPrix
FROM Produit
WHERE Categorie=vcate;

----- (AvgPrix) ;
END;
/
```



A partir d'une requête SQL

Produits qui ne sont pas de la catégorie 'A'
ayant un prix supérieur au prix moyen
des produits de la catégorie 'A'

```
SELECT RefProd, Designation FROM Produit  
WHERE Cat != 'A' AND Prix > PrixMoyen ('A');
```

A partir d'une procédure ou fonction

```
BEGIN  
...  
CASE Prix_Moyen('A')  
WHEN Prix_Moyen('B') >= 5000 THEN...  
...
```



- Parfois on est appelé à introduire des modifications dans le schéma de la base tels que :
la modification de la structure d'une table, l'ajout de nouvelles tables ou même la suppression de tables.
- Pour permettre aux procédures (fonctions) existantes de prendre en compte ces modifications, il faut les recompiler avec la commande suivante :

```
Alter { function/Procedure } <nom_p> compile ;
```

Exemple

```
Alter procedure reduire_prix compile ;  
Alter function Approvisionner compile ;
```



► Comme tout objet manipulé par oracle, les procédures et les fonctions peuvent être supprimées si le besoin est ressenti.

```
Drop { function/Procedure } <nom_p> ;
```

Exemple

```
Drop procedure reduire_prix ;
```



► Il s'agit d'un sous-programme PL/SQL **nommé et défini** dans la section **DECLARE**.

- Ce sous programme **peut avoir des paramètres**.
- Il adhère aux **mêmes règles de portée et de visibilité que les variables**.
- Il est **réutilisable** et facilite la maintenance des applications.

► Nous distinguons deux types de sous programmes non stockés : les procédures et les fonctions.

```
DECLARE -- bloc emboîtant
  procedure <nom_procédure> [(paramètre,
    [paramètre...])] {IS | AS}
    -- Déclaration locales de variables, constant, ...
  BEGIN
    -- Instructions exécutables
  [EXCEPTION
    -- Gestion des exceptions ]
  END [nom_procédure] ;

...
BEGIN
...
EXCEPTION
...
END ;
```



► Puisque les noms des sous-programmes locaux sont des identificateurs, ils **doivent être déclarés avant qu'ils soient utilisés**.

- Par exemple, si un sous programme A appelle un sous programme B, B doit être déclaré avant A.

► Dans le cas de **références mutuelles** (A appelle B et B appelle A), il faut procéder à une déclaration en avant (forward) de l'un ou même des deux sous programmes:

```
FUNCTION/PROCEDURE    <nom_fonction>    [(paramètre 1,  
[paramètre 2...]))];
```

Exercice

Les sous-programmes



Soit la BD suivante :

ARTICLE (nart, desart, pu, qtes)

CLIENT (ncl, nomcl, adr)

FACTURE (nf, datef, #ncl, mntf)

DET_FACT (#nf, #nart, qtef)

1. Créer une fonction CA_CLT_ART permettant de calculer le chiffre d'affaire d'un client donné et pour un article donné.
2. Créer une fonction CA_CLT permettant de calculer le chiffre d'affaire d'un client donné.
Le chiffre d'affaire n'est d'autre que la somme des montants factures (MNTF).
3. Créer une fonction NB_VENTE permettant de calculer le nombre de ventes d'un article donné.
4. Créer une procédure AUGMENTER permettant d'augmenter le prix unitaire d'un article donné selon les règles suivantes :
 - a) Si l'article était vendu plus que 100 fois alors l'augmentation est de 15%
 - b) Si le nombre de vente est compris entre 50 et 100 fois, alors l'augmentation est de 10%.
 - c) Sinon l'augmentation est de 5%.
5. Refaire la question N°4, mais appliquer ce traitement pour tous les articles.